

Meta-Learning and Automated Reinforcement Learning

The design of reinforcement learning algorithms has historically been a manual, iterative process in which researchers propose update rules, loss functions, and hyperparameter schedules on the basis of theoretical intuition and extensive empirical tuning. Meta-learning, often described as “learning to learn,” offers an alternative paradigm in which an outer optimization loop discovers the learning algorithm itself, while an inner loop applies the discovered algorithm to solve individual tasks. This section traces the evolution of meta-reinforcement learning from its earliest formulations through to modern systems capable of discovering update rules that rival hand-designed methods across large-scale benchmarks. We also situate these advances within the broader landscape of Automated Reinforcement Learning (AutoRL) and discuss the limitations that ultimately motivated the turn toward large language model-based algorithm search.

Foundations of Meta-Reinforcement Learning

Two concurrent works in late 2016 established the meta-RL paradigm. Duan et al. [Duan2016] proposed RL², in which a recurrent neural network (RNN) receives observations, actions, rewards, and termination signals as input, retaining its hidden state across episodes within a given Markov decision process. The weights of the RNN are trained slowly via a general-purpose RL algorithm across a distribution of tasks, while the activations of the network encode a fast adaptation procedure for each new task encountered at test time. Evaluated on multi-armed bandits and tabular MDPs, RL² approached the performance of Bayes-optimal strategies without any explicit Bayesian computation. Independently, Wang et al. [Wang2016] at DeepMind introduced a closely related framework in “Learning to Reinforcement Learn,” demonstrating through seven proof-of-concept experiments that a recurrent meta-learner trained with one RL algorithm can implement an entirely separate learned RL procedure in its recurrent dynamics, one configured to exploit structure in the training domain. Together, these two papers showed that the boundary between “algorithm” and “learned behavior” could be dissolved: the policy network does not merely solve tasks but implements the learning algorithm itself.

Model-Agnostic Meta-Learning (MAML), introduced by Finn, Abbeel, and Levine [Finn2017], took a complementary approach by operating directly in parameter space rather than encoding adaptation in recurrent activations. MAML optimizes a neural network’s initial parameters such that a small number of gradient steps on a new task’s loss yields strong generalization. Because MAML makes no assumptions about the model architecture, it applies uniformly to supervised classification, regression, and reinforcement learning. Its elegance, however, comes at a computational cost: the outer loop requires differentiating through the inner optimization, producing second-order gradients that scale poorly with the number of inner steps and parameters. This tension between generality and tractability has been a recurring theme in meta-learning research.

Automating the Components of RL Algorithms

Whereas RL², Wang et al., and MAML meta-learn entire adaptation procedures or initializations, subsequent work asked a more targeted question: can individual components of an RL algorithm, such as the discount factor, the bootstrapping parameter, or the loss function itself, be learned rather than hand-specified? Xu, van Hasselt, and Silver [Xu2018] introduced meta-gradient RL, in which the return function is treated as a parametric object whose meta-parameters, including the discount factor γ and the bootstrapping parameter λ , are adjusted

online via gradient descent on a meta-objective. By differentiating the agent’s performance with respect to these meta-parameters, the algorithm dynamically adapts the nature of the return over the course of training, yielding improvements on the Atari benchmark without manual tuning of these quantities.

Houthoof et al. [Houthoof2018] proposed Evolved Policy Gradients (EPG), which meta-learns a differentiable loss function parameterized by temporal convolutions over the agent’s experience. Rather than backpropagating through the inner loop, EPG uses evolution strategies to optimize the meta-loss, sidestepping the second-order gradient difficulties of MAML-style approaches. The resulting evolved loss enabled faster learning on randomized continuous control tasks and demonstrated superior out-of-distribution generalization compared to RL² and MAML, though it was outperformed by RL² on in-distribution tasks. Kirsch, van Steenkiste, and Schmidhuber [Kirsch2020] extended this direction with MetaGenRL, which distills the experiences of many agents to meta-learn a compact neural objective function. By leveraging off-policy second-order gradients during meta-training, MetaGenRL achieved strong sample efficiency and, notably, generalized to environments that were entirely different from those seen during meta-training, in some cases outperforming human-engineered RL algorithms.

Discovering Complete RL Algorithms

The most ambitious line of meta-RL research aims not merely to tune components but to discover entire RL algorithms from scratch. Oh et al. [Oh2020] introduced Learned Policy Gradient (LPG) at NeurIPS 2020, in which a neural network parameterizes the loss function used to update the agent’s policy. The meta-learner jointly discovers what the agent should predict and how those predictions should drive policy improvement. Remarkably, LPG independently rediscovered an analogue of value functions and a bootstrapping mechanism, and when trained solely on simple toy environments, generalized to achieve non-trivial performance on complex Atari games.

Co-Reyes et al. [CoReyes2021] at Google Brain approached the same goal from a different angle at ICLR 2021 with “Evolving Reinforcement Learning Algorithms.” Rather than parameterizing loss functions with neural networks, they represented the loss as a symbolic computational graph and applied regularized evolution, a variant of genetic programming, to search over graph structures. When learning from scratch on simple gridworld and classical control tasks, the method independently rediscovered temporal-difference learning. When bootstrapped from DQN, the evolved algorithms generalized across classical control, gridworld, and Atari domains, offering the additional advantage of producing interpretable, closed-form loss expressions.

The culmination of this line of work is DiscoRL, published by Oh et al. in Nature in 2025 [Oh2025]. Building on the LPG framework, DiscoRL meta-learns update rules from the cumulative experience of a population of agents across large-scale environment distributions. Two principal algorithms emerged from this process. Disco57, trained on the Atari-57 suite, generalized zero-shot to unseen ProcGen and DMLab-30 domains. Disco103, trained on the expanded union of Atari-57, ProcGen, and DMLab-30, was further evaluated on entirely held-out benchmarks including Crafter, NetHack, and Sokoban. Both variants surpassed all existing hand-designed algorithms on the Atari benchmark, outperforming Rainbow DQN, DQN, PPO, and A3C, methods that required decades of human ingenuity to develop. DiscoRL additionally matched MuZero’s final performance with approximately 40% less computation, establishing that meta-learned algorithms can compete with the most sophisticated planning-based approaches.

The AutoRL Landscape and Its Limitations

Parker-Holder et al. [ParkerHolder2022] provided a comprehensive survey of the Automated Reinforcement Learning (AutoRL) field, proposing a taxonomy that spans hyperparameter optimization, neural architecture search, algorithm selection, and the meta-learning of update rules discussed above. The survey highlighted both the promise and the fragility of RL: agent performance is often acutely sensitive to design choices, and the combinatorial space of possible configurations makes manual tuning impractical at scale.

Despite the impressive results of systems like DiscoRL, the meta-learning paradigm for algorithm discovery faces persistent limitations. First, meta-learned algorithms tend to overfit to their training task distribution. Disco57 generalizes well to ProcGen and DMLab-30, but the extent to which any meta-learned rule can transfer to fundamentally novel problem structures remains an open question. Second, the computational cost is extreme: training DiscoRL requires running populations of agents across hundreds of environments for billions of frames, placing it beyond the reach of most research groups. Third, neural-network-parameterized update rules are opaque. Unlike the symbolic expressions produced by Co-Reyes et al. [CoReyes2021], the learned rules of LPG and DiscoRL resist human interpretation, making it difficult to extract scientific insight or to diagnose failure modes. These three challenges, distributional overfitting, prohibitive compute, and opacity of discovered algorithms, collectively motivated the search for alternative discovery mechanisms. The emergence of large language models as program synthesizers, capable of generating and refining readable code, offered a promising resolution: algorithms expressed as human-interpretable source code, searched over by LLM-guided evolutionary processes that leverage natural language reasoning rather than opaque gradient-based meta-optimization.

References

1. Y. Duan, J. Schulman, X. Chen, P. L. Bartlett, I. Sutskever, and P. Abbeel, “RL²: Fast Reinforcement Learning via Slow Reinforcement Learning,” arXiv:1611.02779, 2016.
2. J. X. Wang, Z. Kurth-Nelson, D. Tirumala, H. Soyer, J. Z. Leibo, R. Munos, C. Blundell, D. Kumaran, and M. Botvinick, “Learning to Reinforcement Learn,” arXiv:1611.05763, 2016.
3. C. Finn, P. Abbeel, and S. Levine, “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, pp. 1126–1135, 2017.
4. Z. Xu, H. P. van Hasselt, and D. Silver, “Meta-Gradient Reinforcement Learning,” in *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018.
5. R. Houthoofd, Y. Chen, P. Isola, B. Stadie, F. Wolski, J. Ho, and P. Abbeel, “Evolved Policy Gradients,” in *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018.
6. L. Kirsch, S. van Steenkiste, and J. Schmidhuber, “Improving Generalization in Meta Reinforcement Learning using Learned Objectives,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.
7. J. Oh, M. Hessel, W. M. Czarnecki, Z. Xu, H. P. van Hasselt, S. Singh, and D. Silver, “Discovering Reinforcement Learning Algorithms,” in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.
8. J. D. Co-Reyes, Y. Miao, D. Peng, E. Real, S. Levine, Q. V. Le, H. Lee, and A. Faust, “Evolving Reinforcement Learning Algorithms,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.
9. J. Parker-Holder, R. Rajan, X. Song, A. Biedenkapp, Y. Miao, T. Eimer, B. Zhang, V. Nguyen, R. Calandra, A. Faust, F. Hutter, and M. Lindauer, “Automated Reinforcement Learning (AutoRL): A Survey and Open Problems,” *J. Artificial Intelligence Research*, vol. 74, pp. 517–568, 2022.
10. J. Oh, G. Farquhar, I. Kemaev, D. A. Calian, M. Hessel, L. Zintgraf, S. Singh, H. van Hasselt, and D. Silver, “Discovering State-of-the-Art Reinforcement Learning Algorithms,” *Nature*, 2025.